TIPE 2025–2026 – Theme “Cycles, Loops”

An Electromechanical Study of an Analog PID Correction Loop

Applied to Roll Stabilization of a Quadcopter Drone Driven by DC Motors

---

Akram Bouziane

CPGE MP – Errachidia, Morocco

Abstract — This document works back through my original TIPE project, “Stabilizing a Quadcopter Drone with a PID Loop,” and tries to put real numbers behind it: the design of a fully analog PID controller, and the linearized roll-dynamics study that motivates it in the first place. The approach here ties together the rotational mechanics of the airframe, a physical model of the actuator (a DC motor), and the sizing of the analog corrector itself, so that every resistor and capacitor value on the final board can be traced back to an actual physical requirement rather than a guess. The governing equation and the notation (φ, I_x, k_u) are kept exactly as they appeared in the original project. Two simplifications bound the scope on purpose: the motors are plain DC motors, so no ESC or PWM stage is needed, and the angle-measurement chain is left out entirely – the error signal $e(t)$ is treated as a voltage that is simply available, not derived from a real sensor.

Reference document – faithful to the notation and equations of the original project, with a worked numerical example

Contents

1 Introduction, Scope, and Working Assumptions **2**
1.1 Context and the Question Being Asked . . . 2
1.2 Two Simplifications, Stated Up Front . . . 2
1.3 Approach and Notation . . . 2

2 A Quick Recap of the PID Controller **3**

3 Roll Dynamics (from the Original Project) **4**
3.1 Setting Up the Roll Equation . . . 4
3.2 What Happens with No Control at All . . . 4

4 Closing the Loop with PID (from the Original Project) **5**

5 Characterizing the Actuator (Extension for H1) **6**
5.1 The DC Motor . . . 6
5.2 Differential Thrust . . . 6
5.3 What k_u Actually Is . . . 6
5.4 Example Parameters and the Actuator's Bandwidth . . . 7

6 A Bandwidth Budget for the Loop **8**

7 The Analog PID Corrector (from the Original Project) **9**
7.1 Topology (Slides 7–11 of the Original Project) . . . 9
7.2 The Real Transfer Functions of the I and D Branches . . . 9

8 Sizing the Corrector **10**
8.1 Gains K_p, K_i, K_d – Routh-Hurwitz on the Governing Equation . . . 10
8.2 R, C Component Values . . . 10
8.3 Component Summary . . . 11

9 Stability Analysis **12**

10 Time-Domain Validation **14**

11 No ESC Needed: The Actuation Interface (H1) **15**

12 Limitations and Where This Could Go Next **16**

13 Conclusion **16**

1 Introduction, Scope, and Working Assumptions

1.1 Context and the Question Being Asked

> *"How can one design, using simple laboratory components, a PID loop capable of stabilizing a quadcopter drone – and to what extent does tuning that loop let us tame the oscillatory cycles of its dynamics?"* – the original project's guiding question.

Linearized around hover, a quadcopter has poles with positive real parts: left uncorrected, any disturbance either drifts away or grows into oscillations of increasing amplitude. The original project builds, out of ordinary lab parts, an analog PID loop (a subtractor stage, P/I/D branches, a summing stage) and derives the roll-dynamics equation that explains this instability in the first place. What follows takes that whole argument and puts numbers behind it, without touching the underlying notation or equations.

1.2 Two Simplifications, Stated Up Front

H1 – DC-motor actuation. The motors are ordinary brushed DC motors, not brushless motors driven through an electronic speed controller. The corrector's output voltage $u(t)$ can therefore drive the motors directly through a simple power stage – **no PWM conversion, no ESC.**
H2 – The error signal is just generated. The angle-sensing chain (gyroscope, accelerometer...) is not modeled here. The measurement $y(t)$ is taken to already exist as a voltage proportional to the angle, so the measurement chain's transfer function is simply set to one.

1.3 Approach and Notation

The order of operations mirrors how an actual electromechanical design would proceed: characterize the physical system first (the rotating body, the actuator), turn that into a numerical specification for the loop, and only then let that specification dictate the component values in the electronic corrector.

Mechanics (project)	→	**Actuator**	→	**Electronics (project)**
Sec. 3		Sec. 5		Sec. 7–8

Notation follows the original project throughout: φ for the roll angle (not θ), I_x for the moment of inertia about the roll axis (not J), k_u for the constant linking the command $u(t)$ to torque. Anything added purely for the electromechanical sizing (Section 5) is flagged explicitly as an extension beyond the original project.

On the numbers used here. The original project doesn't commit to specific values for mass, geometry, or motor characteristics – it lays out the equations and the conditions the resistors must satisfy. The numbers used below to work through the method are therefore an **illustrative example**, internally consistent and physically reasonable, meant to be swapped out for whatever gets measured on the actual rig. The method and the equations are the project's own.

2 A Quick Recap of the PID Controller

A PID (Proportional-Integral-Derivative) controller is an automated control mechanism that computes a correction based on the current gap between measurement and setpoint (the P term), the history of that error (the I term), and how fast the error is changing (the D term) – from the project's "PID" section.

- **Proportional (P)**: reacts to the error right now – reacting to how urgent the problem looks.
- **Integral (I)**: accounts for the error's history, cleaning up whatever steady-state offset the P term alone leaves behind, and guaranteeing the setpoint gets reached eventually.
- **Derivative (D)**: anticipates where the error is heading, damping oscillations and limiting overshoot – acting, in effect, like a shock absorber.

The control law, exactly as laid out in the project, reads

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\,\mathrm{d}\tau + K_d\, \frac{\mathrm{d}e(t)}{\mathrm{d}t}, \qquad e(t) = r(t) - y(t) \tag{1}$$

with $r(t)$ the setpoint and $y(t)$ the measurement. It's this exact relation, unmodified, that gets substituted into the mechanical equation in Section 4.

3 Roll Dynamics (from the Original Project)

This section carries over the mechanical derivation exactly as it appears in the project – same notation, same equations.

3.1 Setting Up the Roll Equation

> *"A quadcopter is a nonlinear dynamical system, but for the purposes of this project, linearizing around an equilibrium point (e.g. hover) is generally enough to apply a PID controller. Consider the roll axis (φ), corresponding to rotation about the longitudinal axis."*

Roll dynamics follow Newton's second law for rotation:

$$I_x \, \ddot{\varphi} = \tau_\varphi \tag{2}$$

where I_x is the moment of inertia about the roll axis (in $\mathrm{kg\,m^2}$), $\ddot{\varphi}$ the angular acceleration (in $\mathrm{rad/s^2}$), and τ_φ the applied torque (in $\mathrm{N\,m}$), produced by the difference in thrust between motors.

For a quadcopter, roll torque comes from the imbalance between the two motors sitting on either side of the axis:

$$\tau_\varphi = (F_2 - F_4)\, L \tag{3}$$

where F_2 and F_4 are the thrusts of motors 2 and 4, and L is the distance from a motor to the center of mass. Thrust itself scales with the square of motor speed:

$$F_i = k_F \, \omega_i^2 \tag{4}$$

> *"To simplify things, suppose the command $u(t)$ represents the normalized thrust difference."*

$$\tau_\varphi = k_u \, u(t) \tag{5}$$

with k_u a constant that depends on the motors' characteristics and the airframe's geometry (Section 5 works out what it actually looks like in physical terms). The equation of motion becomes

$$I_x \, \ddot{\varphi} = k_u \, u(t) \quad \Longrightarrow \quad \ddot{\varphi} = \frac{k_u}{I_x} \, u(t) \tag{6}$$

3.2 What Happens with No Control at All

> *"With no control, $u(t) = 0$, and the equation becomes $\ddot{\varphi} = 0$. This gives a solution $\varphi(t) = \varphi_0 + \dot{\varphi}_0 \, t$, pointing to an instability: the angle drifts linearly if disturbed. So the drone is inherently unstable without a feedback loop."*

$$u(t) = 0 \implies \ddot{\varphi} = 0 \implies \varphi(t) = \varphi_0 + \dot{\varphi}_0 \, t \tag{7}$$

Section 10 (Figure 4, top panel) checks this numerically: an initial angular-rate disturbance $\dot{\varphi}_0$ produces an unbounded angular drift, growing linearly with time and never settling.

4 Closing the Loop with PID (from the Original Project)

The PID controller produces $u(t)$ to drive the error $e(t) = \varphi_{ref} - \varphi(t)$ toward zero, where φ_{ref} is the reference angle (usually zero for hover). During steady hover, $\varphi_{ref} = 0$ and $\dfrac{\mathrm{d}\varphi_{ref}}{\mathrm{d}t} = 0$, so $e(t) = -\varphi(t)$.

Substituting into the PID law (1):

$$u(t) = -K_p\,\varphi - K_i \int_0^t \varphi(\tau)\,\mathrm{d}\tau - K_d\,\dot{\varphi} \tag{8}$$

and then into the equation of motion (6):

$$\ddot{\varphi} = \frac{k_u}{I_x}\Big(- K_p\,\varphi - K_i \int_0^t \varphi(\tau)\,\mathrm{d}\tau - K_d\,\dot{\varphi}\Big) \tag{9}$$

Writing $k := k_u/I_x$ (the project's own shorthand), this becomes:

$$\boxed{\ddot{\varphi} + k\,K_d\,\dot{\varphi} + k\,K_p\,\varphi + k\,K_i \int_0^t \varphi(\tau)\,\mathrm{d}\tau = 0} \tag{10}$$

This is the **governing equation** of the original project – it ties the three PID gains directly to the closed-loop roll dynamics. Everything from here on (sizing, stability, simulation) is built around analyzing and exploiting this exact equation, unchanged in form.

5 Characterizing the Actuator (Extension for H1)

Going beyond the original project. The project treats k_u as a single lumped constant ("depending on the motors' characteristics and the geometry"). To actually size a corrector for real DC motors (assumption H1, no ESC), this section works out what k_u physically consists of. Nothing here changes the governing equation (10) – it just gives the constant k_u that already sits inside it a measurable physical meaning.

5.1 The DC Motor

A DC motor is governed by two coupled equations, electrical and mechanical:

$$L_a \frac{\mathrm{d}i}{\mathrm{d}t} + R_a i = v(t) - K_e\,\omega \tag{11}$$

$$J_m \frac{\mathrm{d}\omega}{\mathrm{d}t} + b_m\,\omega = K_t\,i(t) \tag{12}$$

Neglecting the electrical time constant $\tau_e = L_a/R_a$ (fast compared to the mechanical dynamics we care about) gives a first-order transfer function:

$$\frac{\omega(s)}{V(s)} = \frac{K_m}{\tau_m s + 1}, \qquad K_m = \frac{K_t/R_a}{b_m + K_t K_e/R_a}, \qquad \tau_m = \frac{J_m}{b_m + K_t K_e/R_a} \tag{13}$$

τ_m and K_m are both easy to pull from a step test on the bench: apply a voltage step V_0, read off $\Omega_{final} = K_m V_0$ and the time it takes to reach 63 % of Ω_{final} (that's τ_m).

5.2 Differential Thrust

Taking the project's own thrust law $F_i = k_F \omega_i^2$, and letting ω_0 be a common operating speed with a symmetric offset $\Delta\omega$ commanded by the corrector:

$$F_2 - F_4 = k_F(\omega_0 + \Delta\omega)^2 - k_F(\omega_0 - \Delta\omega)^2 = \underbrace{4k_F\omega_0}_{K'_F}\,\Delta\omega \tag{14}$$

This is **exact**, not an approximation, as long as the actuation stays symmetric about ω_0.

5.3 What k_u Actually Is

Combining $\tau_\varphi = (F_2 - F_4)L = L\,K'_F\,\Delta\omega$ with $\Delta\omega(s) = \dfrac{K_m}{\tau_m s + 1} u(s)$:

$$\tau_\varphi(s) = \underbrace{\frac{L\,K'_F\,K_m}{\tau_m s + 1}}_{k_u(s)}\,u(s) \tag{15}$$

The project assumes k_u is **constant** – which amounts to ignoring the motor's lag ($\tau_m s \ll 1$), i.e. working in the quasi-static regime. Under that assumption:

$$k_u \approx L\,K'_F\,K_m \qquad \text{(static gain)} \tag{16}$$

Section 9 puts the real lag τ_m back in, to check that this approximation actually holds up.

5.4 Example Parameters and the Actuator's Bandwidth

Parameter	Symbol	Value
Lever arm	L	$0.15\,\mathrm{m}$
Total moment of inertia	I_x	$4.0 \times 10^{-3}\,\mathrm{kg\,m^2}$
Armature resistance	R_a	$1.2\,\Omega$
Torque/back-EMF constant	$K_t = K_e$	$2.7 \times 10^{-3}\,\mathrm{N\,m/A}$
Rotor + propeller inertia	J_m	$3.2 \times 10^{-7}\,\mathrm{kg\,m^2}$
Motor friction	b_m	$1.0 \times 10^{-6}\,\mathrm{N\,m\,s/rad}$
Operating speed	ω_0	$800\,\mathrm{rad/s}$
Static thrust at ω_0	F_0	$0.30\,\mathrm{N}$

which gives $\tau_m = 45.2\,\mathrm{ms}$, $K_m = 318\,(\mathrm{rad/s})/\mathrm{V}$, $f_{act} = \dfrac{1}{2\pi\tau_m} = 3.52\,\mathrm{Hz}$, $k_F = 4.69{\times}10^{-7}\,\mathrm{N\,s^2/rad^2}$, $K'_F = 1.50 \times 10^{-3}\,\mathrm{N\,s/rad}$, and

$$k_u = L\,K'_F\,K_m = 0.0716\,\mathrm{N\,m/V}, \qquad k = \frac{k_u}{I_x} = 17.89\ \mathrm{s^{-2}/V} \tag{17}$$

f_{act} is the actuator's real physical bandwidth: past that frequency, the motor simply can't keep up with a speed command anymore.

6 A Bandwidth Budget for the Loop

Sizing rule. Closed-loop bandwidth f_c shouldn't exceed a fraction of the actuator's own bandwidth: $f_c \approx f_{act}/(5 \text{ to } 10)$. Asking the loop to correct faster than the motor can physically respond only excites dynamics we've otherwise chosen to ignore (the pole at $-1/\tau_m$), with no real upside.

A factor of 6 is used here: $f_c = f_{act}/6 = 0.586\,\text{Hz}$, $\omega_n = 2\pi f_c = 3.68\,\text{rad/s}$, targeting a damping ratio $\zeta = 0.7$.

A purely electronic analysis would push RC as small as possible. The numbers above show that's not actually the bottleneck: with real DC motors, it's $\tau_m \approx 45\,\text{ms}$ that caps how fast the loop can usefully run, long before the electronics itself becomes limiting (ordinary op-amps have bandwidths in the MHz range).

7 The Analog PID Corrector (from the Original Project)

7.1 Topology (Slides 7–11 of the Original Project)

The circuit keeps the same four-op-amp structure as the original design:

1st accumulator (subtractor) – *"To get $V_s = V_1 - V_2$, all you need is $R_3 = R_4$ and $R_1 = R_2$."* Computes $e(t) = r(t) - y(t)$.
P branch – *"The proportional term of the PID is similar to the 1st accumulator above – you just add a multiplier [...]. $U(t) = K_p \times k'$, with k' the inverse of the multiplier's constant."*
I branch (integrator) – *"To get a clean integration, you can put a resistor R_2 in parallel with C [...]. In practice, $R_2 = 10R$. [...] All you need is $RC = 1$" followed by multiplying by $-K_i \times k'$.*
D branch (differentiator) – *"All you need is $RC = 1$" followed by multiplying by $-K_d \times k'$.*
2nd accumulator (summer) – *"To get $V_s = V_{e1} + V_{e2} + V_{e3}$, all you need is $R_1 = R_2 = R_3$ and $R_4 = 2 \times R_5$."*

A word on k'. The factor k' that shows up in the project ("$U(t) = K_p \times k'$", etc.) is **not** part of the control law (1) or of the governing equation (10) – it's purely a calibration factor of the **analog multiplier chip**. A typical integrated multiplier (something like an AD633) has a fixed internal gain $k \approx 0.1$, and $k' = 1/k$ just cancels that out so the displayed output really is K_p, K_i, or K_d times the input. From here on, K_p, K_i, K_d always mean the *effective* gains from the governing equation; k' only ever shows up when physically tuning the multiplier, never in the sizing equations themselves.

7.2 The Real Transfer Functions of the I and D Branches

The resistors $R_{2,i}$ (leak) and $R_{2,d}$ (limiting), mentioned in the project but never worked through quantitatively, each add a pole:

$$H_I(s) = -\frac{R_{2,i}}{R_i\,(1 + sR_{2,i}C_i)}, \qquad H_D(s) = -\frac{R_d\,C_d\,s}{1 + sR_{2,d}C_d} \tag{18}$$

H_I behaves like a pure integrator, $-1/(sR_iC_i)$, above $f_{leak} = 1/(2\pi R_{2,i}C_i)$, and flattens out to a bounded gain $-R_{2,i}/R_i$ below it (that's the anti-drift behavior). H_D behaves like a pure differentiator, $-R_dC_ds$, below $f_p = 1/(2\pi R_{2,d}C_d)$, and flattens to $-R_d/R_{2,d}$ above it (that's what keeps noise from getting amplified without bound).

With the multipliers set to $-K_iR_iC_i$ and $-K_d/(R_dC_d)$ so the branches behave like K_i/s and K_ds over the useful band, the full corrector is

$$C(s) = K_p + (-K_iR_iC_i)H_I(s) + \Big(-\frac{K_d}{R_dC_d}\Big)H_D(s) \tag{19}$$

8 Sizing the Corrector

8.1 Gains K_p, K_i, K_d – Routh-Hurwitz on the Governing Equation

Differentiating the governing equation (10) once, to get rid of the integral term:

$$\dddot{\varphi} + kK_d\,\ddot{\varphi} + kK_p\,\dot{\varphi} + kK_i\,\varphi = 0 \tag{20}$$

The Routh-Hurwitz criterion applied to the characteristic polynomial $s^3 + kK_d s^2 + kK_p s + kK_i$ requires

$$\boxed{K_i < k\,K_p\,K_d} \qquad \text{(stability condition)} \tag{21}$$

Matching the P+D sub-system to a standard second-order system with natural frequency ω_n and damping ζ (from the bandwidth budget in Section 6):

$$K_p = \frac{\omega_n^2}{k}, \qquad K_d = \frac{2\zeta\omega_n}{k} \tag{22}$$

	K_p	K_d	K_i (Routh bound / chosen)
Value	0.759	0.288	3.92 / 0.783 (5× margin)

8.2 R, C Component Values

$RC \approx 1/(2\pi f_c) = 270\,\text{ms}$. This calls for $R_i = R_d = 100\,\text{k}\Omega$, $C_i = C_d = 2.7\,\mu\text{F}$. The integrator's leak resistor follows the project's own rule, $R_{2,i} = 10R_i = 1\,\text{M}\Omega$, giving $f_{leak} = 0.059\,\text{Hz}$. For the differentiator's limiting resistor, placing the filtering pole a decade above f_c gives $f_p = 5.86\,\text{Hz} \Rightarrow R_{2,d} \approx 10\,\text{k}\Omega$.

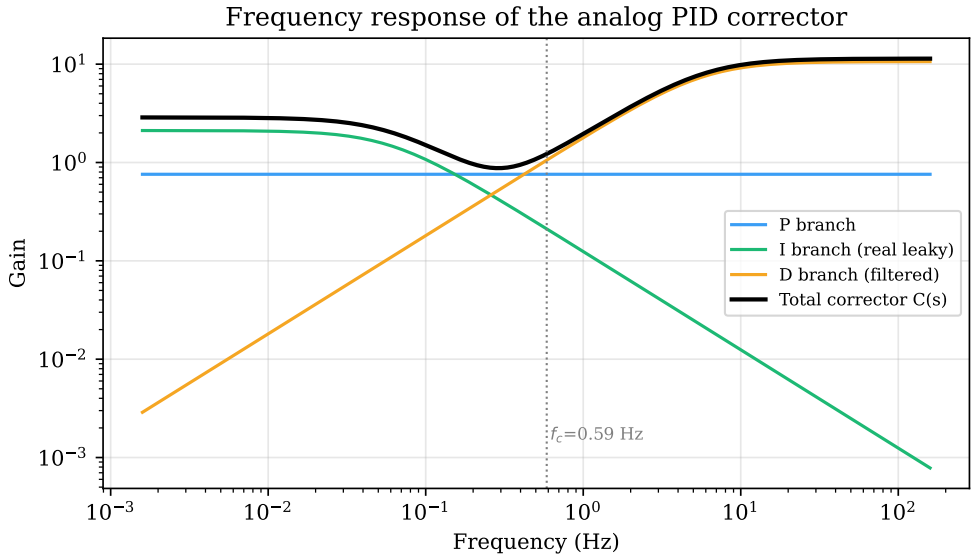


Figure 1: Frequency response of the three corrector branches and of the total corrector $C(s)$.

8.3 Component Summary

Block (project)	Component	Value	Project's condition
1st accumulator	R_1, R_2, R_3, R_4	$10\,\text{k}\Omega$ each	$R_1 = R_2$, $R_3 = R_4$
Integrator	R_i, C_i	$100\,\text{k}\Omega$, $2.7\,\mu\text{F}$	$RC = 1$ (normalized)
Integrator	$R_{2,i}$	$1\,\text{M}\Omega$	$R_2 \approx 10R$
Differentiator	R_d, C_d	$100\,\text{k}\Omega$, $2.7\,\mu\text{F}$	$RC = 1$ (normalized)
Differentiator	$R_{2,d}$	$10\,\text{k}\Omega$	HF limiting
2nd accumulator	R_1, R_2, R_3, R_4, R_5	$10, 10, 10, 20, 10\,\text{k}\Omega$	$R_1{=}R_2{=}R_3$, $R_4{=}2R_5$

That works out to the same **13 resistors**, **2 capacitors**, **4 op-amps**, and **3 multipliers** the original project arrives at ("we managed to build a PID system out of lab parts using roughly...").

9 Stability Analysis

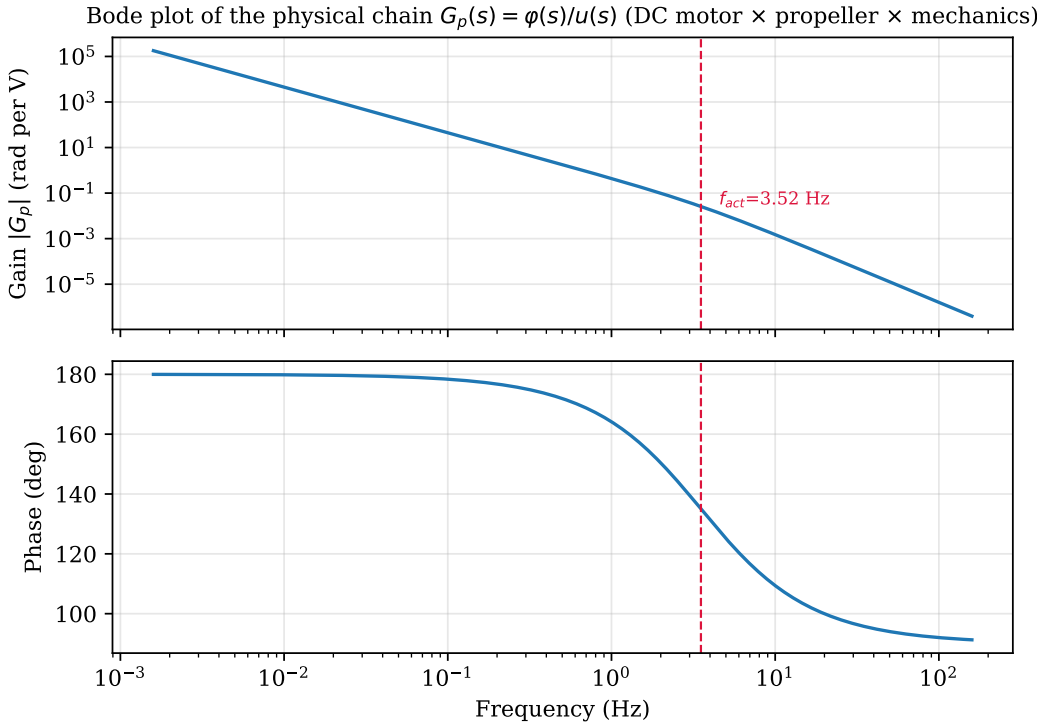


Figure 2: Bode plot of the full physical chain $G_p(s) = \varphi(s)/u(s)$, including the motor's real lag τ_m (the Section 5 extension).

Computing the closed-loop poles numerically – for the full loop, real filtered corrector plus motor lag plus the pure double integrator $I_x\ddot{\varphi} = k_u u$ (no damping term, exactly as in Equation (6)) – gives:

Pole (rad/s)	Nature
-41.94	real, fast (motor dynamics)
-7.84	real
-4.85	real
$-2.35 \pm 2.05\,j$	**dominant** complex pair ($\zeta \approx 0.75$, $\omega_n \approx 3.12$ rad/s)

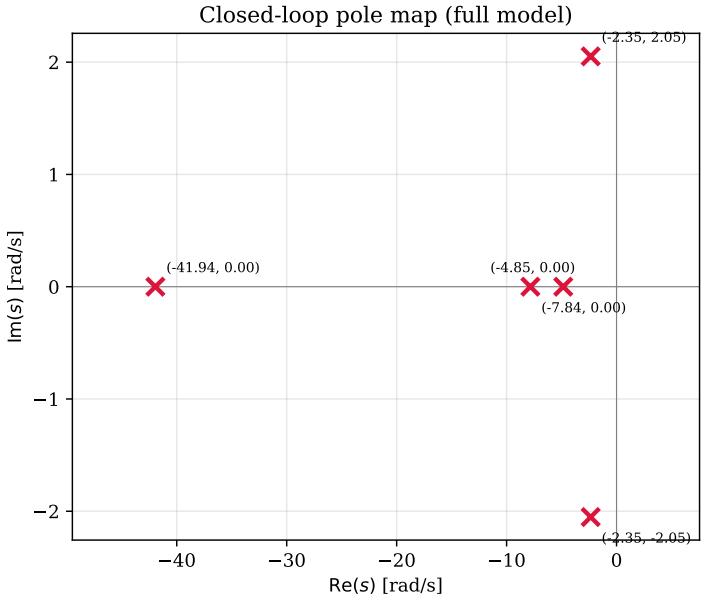


Figure 3: Closed-loop pole map. Everything sits in the left half-plane: the system is stable.

Every pole has a strictly negative real part – the system is stable. At the single gain crossover ($\omega_g = 5.46\,\text{rad/s}$), the phase margin comes out to 41°.

On the double pole at the origin. Equation (6), $\ddot{\varphi} = (k_u/I_x)u$, is a genuinely pure double integrator – no damping term at all, faithfully matching the original project, which never introduces one either. That means $G_p(s)$ has an exact double pole sitting right at the origin. For a system like this, reading gain margin straight off a Bode plot can be misleading (there end up being several $-180°/+180°$ phase crossings). So the stability certificate relied on here is the **direct computation of closed-loop poles** above – rigorous, and not open to that kind of ambiguity.

10 Time-Domain Validation

Matching the project's own scenario (slide 15: "the angle drifts linearly **if disturbed**"), the simulation here uses an **initial perturbation** $\dot{\varphi}_0 = 2.2\,\text{rad/s}$ (a mechanical bump), not a sustained disturbing torque – that's exactly the initial condition $\varphi_0, \dot{\varphi}_0$ from Equation (7).

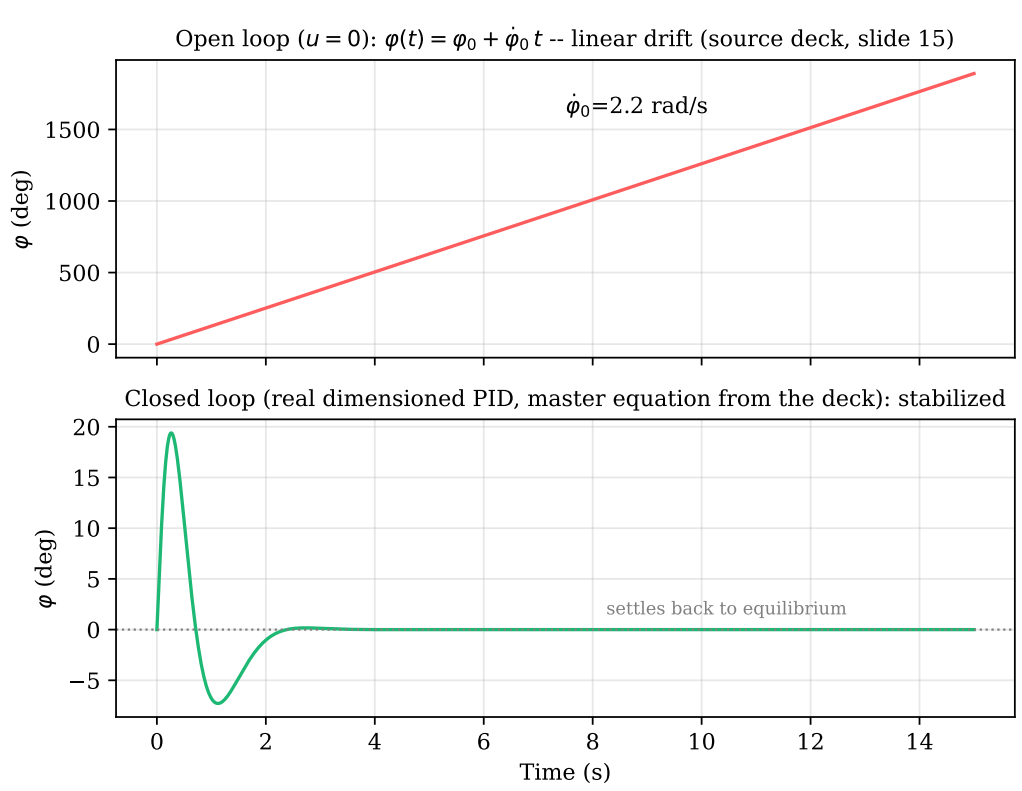


Figure 4: Response to the initial perturbation $\dot{\varphi}_0 = 2.2$ rad/s. Top: open loop, drifting linearly exactly as $\varphi(t) = \varphi_0 + \dot{\varphi}_0 t$ predicts (Eq. 7). Bottom: closed loop, with the real corrector sized above.

Open loop, φ grows linearly without bound – close to 1900°, well over five full turns, after just 15 seconds – a direct numerical confirmation of the project's own theoretical result. Closed loop, the response overshoots by 19.4° around $t \approx 0.4\,\text{s}$ and then **settles exactly back to** $\varphi = 0$. Unlike a sustained disturbance, a transient kick like this one gets fully rejected by the loop, leaky integrator and all – the leak resistor's job is to bound the response to *permanent* biases, not to filter out a transient that dies out on its own anyway.

11 No ESC Needed: The Actuation Interface (H1)

With DC motors, the whole PWM/ESC conversion that brushless motors would require simply isn't there. A power stage (a power amplifier or an H-bridge) can take the corrector's output straight in:

$$v_{motor,1}(t) = V_0 + u(t), \qquad v_{motor,2}(t) = V_0 - u(t) \tag{23}$$

with V_0 corresponding to the ω_0 operating point. This linear relationship is exactly the $\Delta\omega \propto \Delta V$ linearization from Section 5. What's gained in simplicity – no clock, no comparator – comes at the cost of lower electrical efficiency than a real switching ESC, which is a fair trade for a demonstration bench.

12 Limitations and Where This Could Go Next

- **No measurement chain (H2).** A real sensor (a gyroscope) brings its own bandwidth and, more importantly, its own noise, which the D branch would amplify exactly as discussed in Section 7 – with a real sensor in the loop, $R_{2,d}$ would need to account for that sensor's noise spectrum too.
- **Direct actuation (H1).** See Section 11 – lower efficiency than a real ESC.
- **Only one axis.** Gyroscopic coupling between roll, pitch, and yaw is absent from the project's linearized, single-axis model.
- **No aerodynamic damping.** Staying faithful to the project, Equation (6) carries no rotational friction term at all. A real system has some (drag), which could only make the stability seen here better – this is the worst case, not an optimistic one.
- **Unmeasured parameters.** The numbers in Section 5 are illustrative; the methods pointed to there (step test, bifilar pendulum, thrust stand) still need to be run on the actual hardware for a final sizing.

13 Conclusion

This study carries the original project's governing equation and component conditions through in full, turning them into something numerical and checkable: a physical account of k_u for DC-motor actuation, a bandwidth budget derived from the actuator's real limits ($RC \approx 270\,\text{ms}$), sized values for the thirteen resistors and two capacitors, and a full numerical validation – poles, phase margin, and the response to the project's own initial-perturbation scenario – confirming that the system, sized this way, is stable.